SUMMARY

Datapoints:

M+N	Time in MS (Basic)	Time in MS (Efficient)	Memory in KB (Basic)	Memory in KB (Efficient)
16	0.078	0.164	17968	18176
64	0.244	0.581	17904	18288
128	0.613	1.316	18112	18224
256	2.399	3.997	18608	18336
384	5.174	8.533	19392	18400
512	8.754	15.526	20608	18288
768	21.189	32.222	24272	18544
1024	37.003	58.516	28864	18432
1280	62.148	90.154	34752	18432
1536	81.887	125.780	42224	18720
2048	155.671	233.176	62128	18560
2560	239.965	363.039	85984	18752
3072	338.499	507.425	114112	18896
3584	493.027	722.552	148064	19056
3968	592.104	867.748	173856	19072

Insights:

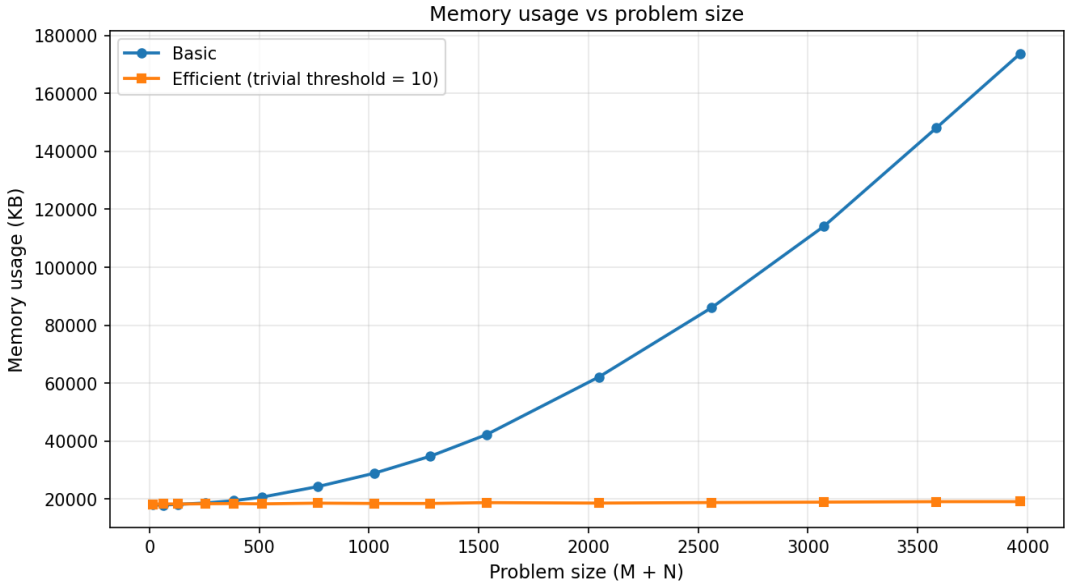
As M+N increases, the Basic algorithm's memory grows drastically from 17,968 KB at M+N=16 to 173,856 KB at M+N=3968. The Efficient algorithm's memory remains nearly constant throughout (18,176 KB to 19,072 KB), demonstrating its O(M+N) space advantage.

The Efficient algorithm is consistently slower than Basic. At M+N=3968, Efficient takes approximately 868 ms. while Basic takes approximately 592 ms. confirming that memory savings come at a time cost due to recursive overhead.

For small problem sizes (M+N ≤ 128), both algorithms perform comparably in both time and memory, suggesting the divide-and-conquer overhead is negligible at small scales.

The crossover in practical usefulness occurs at larger problem sizes where Basic's memory becomes prohibitive, making Efficient the only viable option despite the slower runtime.

Graph 1 – Memory vs Problem Size (M+N)



Graph 1 – Memory vs Problem Size (M+N)

Nature of the Graph (Logarithmic / Linear / Polynomial / Exponential)

Basic: Polynomial (quadratic)

Efficient: Linear

Explanation:

Graph 1 plots memory usage against problem size for both the basic and efficient algorithms. The two curves diverge dramatically as problem size increases, highlighting the memory cost of the basic approach.

The basic algorithm uses dynamic programming to assess the cost of all possible sequence alignments between two strings X[0…i] and Y[0…j]. A two-dimensional DP array of size m·n is required to maintain all computed costs; therefore the basic algorithm has a space complexity of O(m·n). In Graph 1, the basic algorithm has a clear parabolic growth, confirming the O(m.n) polynomial space complexity. At small problem sizes (M+N < 500), both algorithms consume similar memory (19,000 KB), as the table is still small. However, beyond this point the basic algorithm's memory climbs steeply, reaching approximately 173,856 KB at M+N = 3968 almost nine times the baseline.

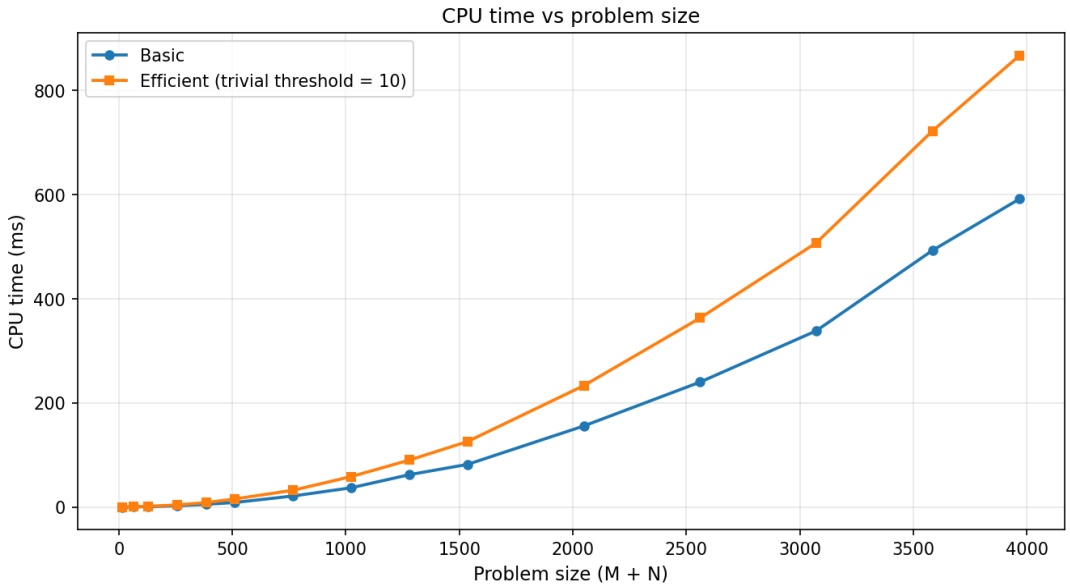
In the efficient algorithm, we use a Divide and Conquer approach in which dynamic programming is used to find the optimal division at the "divide" stage. As a result, only two DP columns are required to assess all possible combinations for some split X_i ($0 \le i \le n$) and Y_j ($0 \le j \le m$). Therefore, the overall space complexity of this algorithm is O(m + n).

Another important consideration in the efficient algorithm is the selection of the trivial problem size threshold. Please refer to Appendix A for details on how a value of 10 was selected for this threshold.

In Graph 1, we see the efficient algorithm's memory usage appears almost flat; however, on closer inspection, it does exhibit linear growth. One explanation for the relative flatness comes from the memory overhead required to run the Python base process. Problem sizes as large as 4000 still consume relatively little memory compared to the base process, which is virtually the same overhead for all problem sizes.

The crossover where the two algorithms diverge noticeably occurs around M+N ≈ 500, beyond which the quadratic memory cost of the basic algorithm becomes increasingly prohibitive. This graph demonstrates that for large problem sizes, the efficient algorithm provides a substantial practical memory advantage while solving the same problem.

Graph 2 – Time vs Problem Size (M+N)



Graph 2 – Time vs Problem Size (M+N)

Nature of the Graph (Logarithmic / Linear / Polynomial / Exponential)

Basic: Polynomial

Efficient: Polynomial

Explanation:

The time complexity for the basic algorithm is dominated by filling the m × n DP array with cost computations between all possible alignments in X[0…i] and Y[0…j], which is done in O(mn) time. Additionally, the cost of each subproblem involves a simple comparison with neighbours and a lookup in the delta mismatch matrix, all of which is done in O(1) time. In order to verify the overall time complexity, consider the cost of each step in the algorithm:

1. Initializing the DP array in O(m.n) time
2. Filling the base cases (involving only computations on the 0-row and 0-column) at a cost of O(m + n)
3. Computing the recurrence formula in O(m.n)
4. One top-down backtracking pass to extract the optimally aligned strings, which is done in O(m.n)

Referring to Graph 2, the basic algorithm's time cost has a polynomial form, confirming the overall O(m.n) time complexity.

The efficient algorithm's recursive Divide and Conquer approach has the same overall complexity of O(m.n), but the work done at each subproblem is slightly higher. Again, we consider the cost of each step in the algorithm to verify the overall complexity:

1. Splitting X at a midpoint is O(1) time, as it is only a computation on array index
2. The forward and backwards DP passes to determine the optimal split between X and Y each require a cost O((m/2) · n)
3. The divide and conquer recursion (the "conquer" step) on (X_L, Y_0…k) and (X_R, Y_k…n) requires T(m/2, k) + T(m/2, n − k)

For the efficient algorithm, it is necessary to understand that the work at each subproblem is a constant factor greater than the basic algorithm. Consider a subproblem at depth d where we align a substring:

1. X of size m/2^d (X is always halved at the midpoint)
2. Y of varying size, but at this recursion depth, all Y subproblems still add to n

Therefore the work per subproblem is expressed as:

$$c \cdot (m / 2^d) \cdot n = cm.n / 2^d$$

Overall, the sum of this geometric series for all subproblems shows the total work for the divide and conquer algorithm is 2mn. Recall the basic DP algorithm fills the m × n matrix one time at a cost of m.n.

To conclude, both the efficient and basic algorithms have a time complexity of O(m.n); however, the efficient algorithm has a 2× constant factor overhead. At M+N = 3968, the basic algorithm completes in approximately 592 ms while the efficient algorithm requires approximately 868 ms — making it roughly 1.4 times slower despite solving the same problem. Inspecting Graph 2, it is clear that both algorithms have the same polynomial order, but the efficient algorithm is always higher than the basic algorithm by some constant factor.

The trivial threshold of 10 partially mitigates this overhead by solving subproblems of size ≤ 10 directly, without further recursive decomposition. This avoids unnecessary recursion at the bottom of the call tree and improves practical runtime. Without this threshold, the performance gap between the two algorithms would likely be more pronounced. Please refer to Appendix A for details on how a value of 10 was selected for this threshold.

Taken together, Graphs 1 and 2 paint a complete picture: the efficient algorithm offers a substantial memory advantage at the cost of additional computation time — a trade-off that must be carefully considered depending on the constraints of the problem at hand.

Contribution

(Please mention what each member did if you think everyone in the group does not have an equal contribution, otherwise, write "Equal Contribution")

5892633819, 9385407385, 9393761377, 1253387049: Equal Contribution

Appendix A – Selection of the Trivial Size Threshold

The trivial size threshold is chosen based on memory performance, since the efficient algorithm aims to optimize memory usage at the cost of higher time complexity.

To determine the optimal trivial size threshold, we conducted a sweep across threshold values of {1, 10, 100, 500, 1000, 5000, 10000, 50000, 100000, 500000} and ran the Efficient algorithm on all 15 datapoint inputs for each threshold value.

Figure 1 below shows the memory usage at each problem size for the efficient algorithm using this sweep of threshold values. Based on these initial results, values ≥ 50,000 were excluded from further consideration.

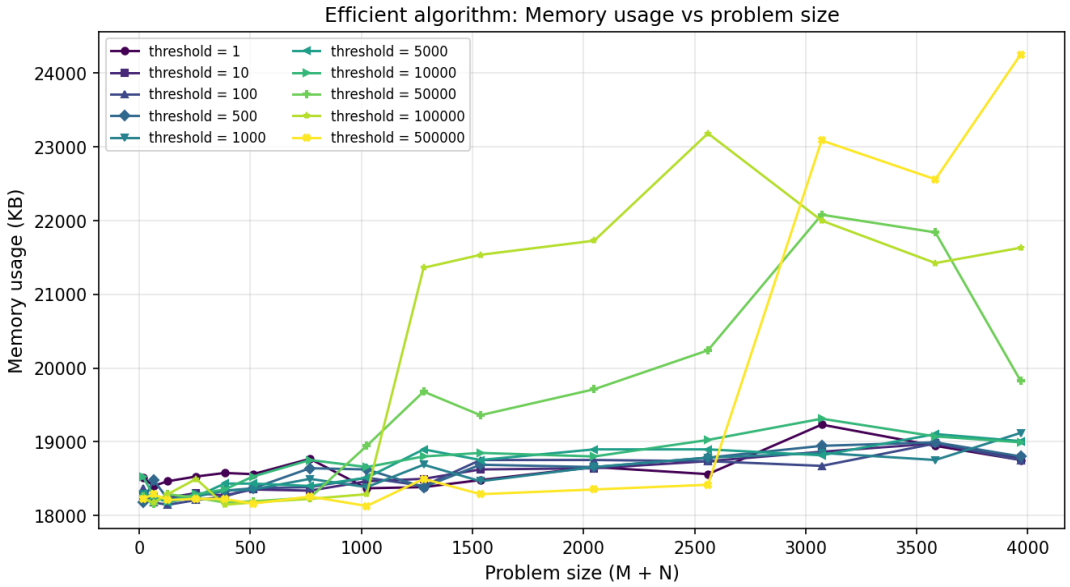


Figure 1. Memory (KB) at each problem size for thresholds (1…500,000)

Threshold values between 1 and 10,000 have relatively similar growth and are visually difficult to distinguish a clear "winner." Therefore, an "average rank" metric is computed (Figure 2): at each problem size, the corresponding memory points for each threshold are ranked 1 to 7, where 1 is the best memory performance and 7 is the worst. These rankings are then averaged across all problem sizes to determine which threshold value tends to produce the best results.

Results:

Threshold	Avg Memory Rank
10	2.70 (best)
1000	3.00
100	3.17
500	3.57
1	4.87
10000	5.20
5000	5.50

As shown in the chart below, threshold = 10 achieved the best average memory rank of 2.70, meaning it most consistently used the least memory across all problem sizes. This makes it the optimal trivial size, it minimizes the base case subproblem size, ensuring the divide-and-conquer approach recurses as deeply as possible before falling back to basic DP, thus preserving the O(m+n) memory guarantee. Based on the outcome of the average rank analysis, the threshold of 10 was selected for the trivial problem size threshold.

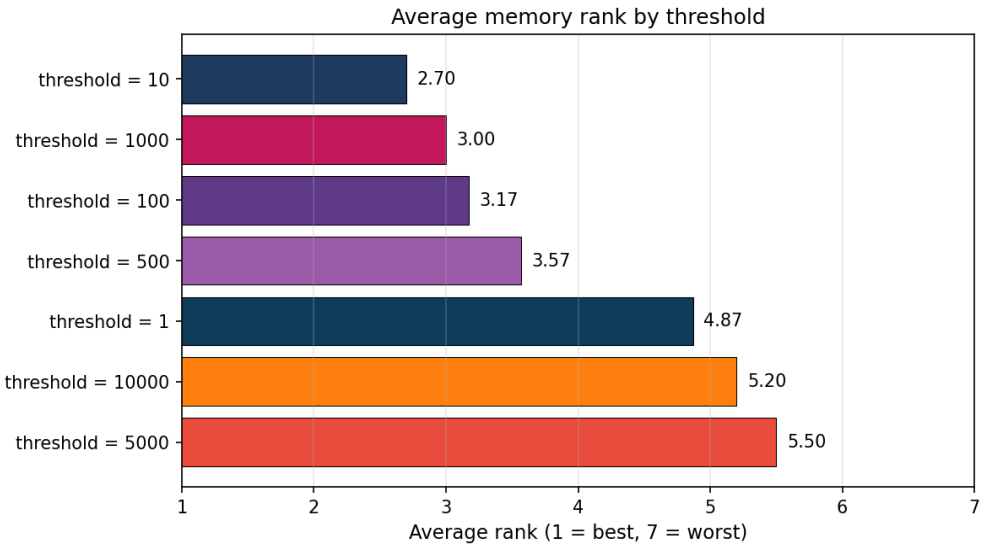


Figure 2. Average Memory Rank by Threshold (over all problem sizes)